

+

Informatik 2 – Dateiformate & Binärdateien

K

A



©image: various

Heute

- Bildformate
- Textdatei
- Binärdatei
- BMP Dateiformat

C++

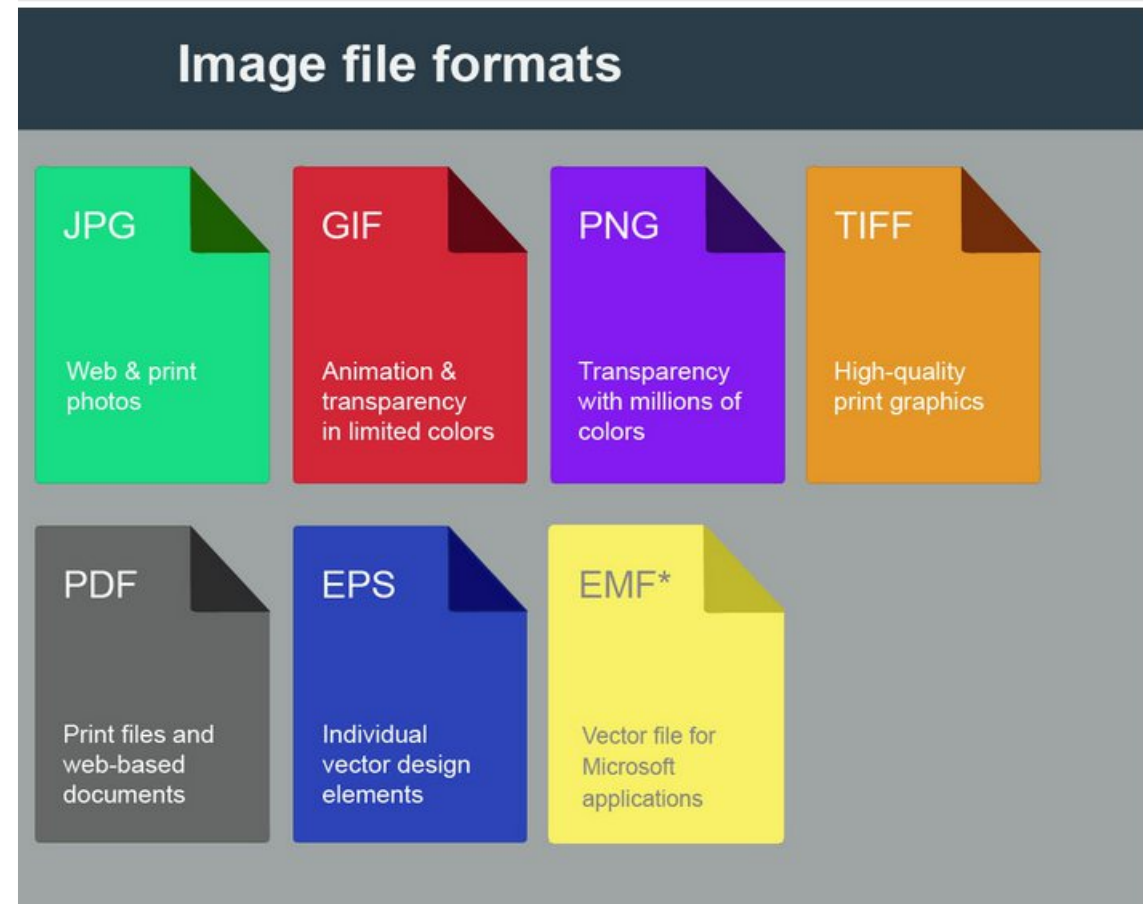
- Implicite cast
- Explicite cast
- Reinterpret cast
- Binary data i/o

Verschiedene Dateiformate für Bilder

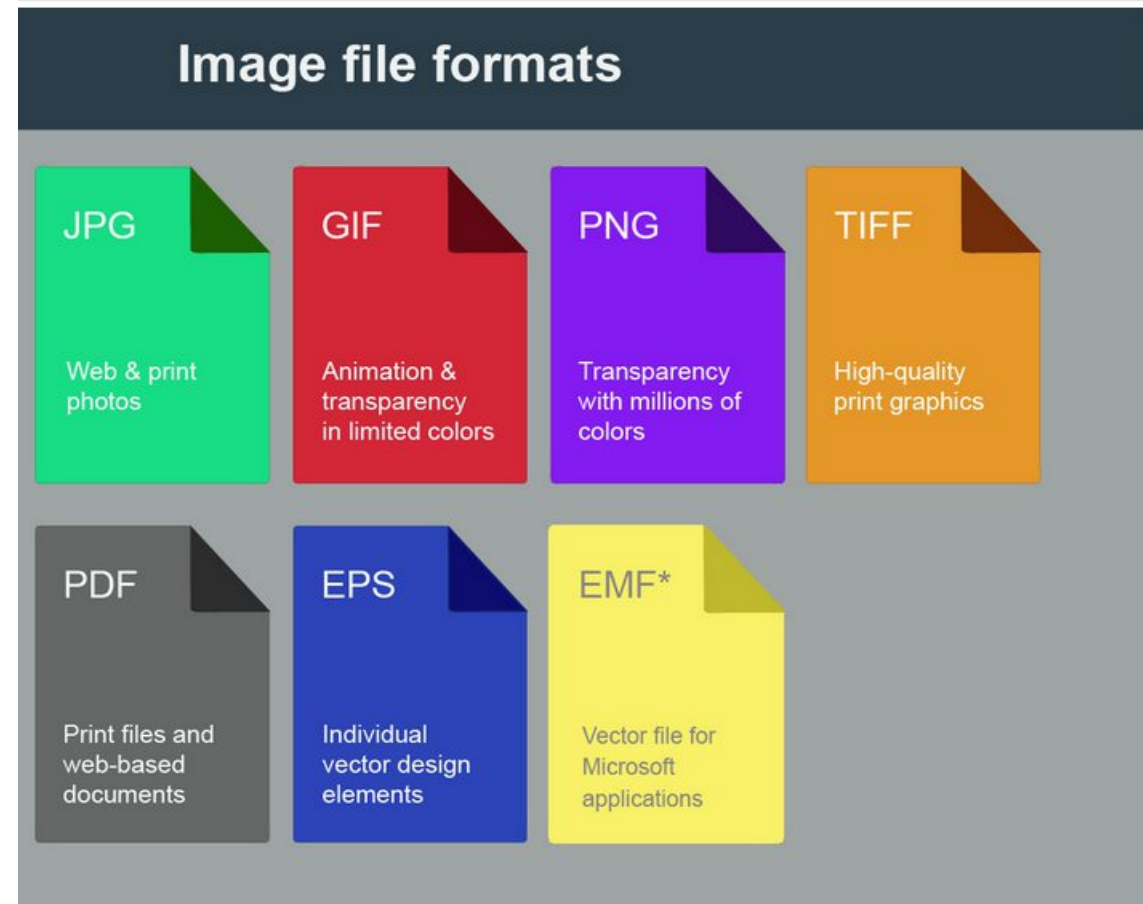
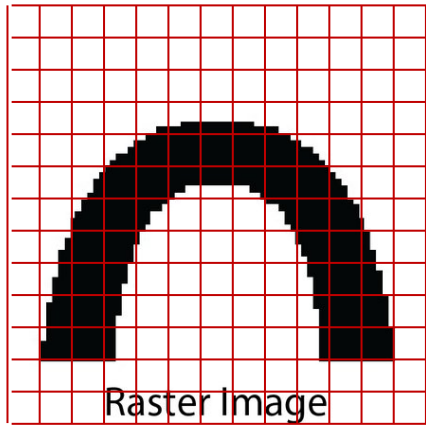


Welche kennen Sie?

Verschiedene Dateiformate für Bilder



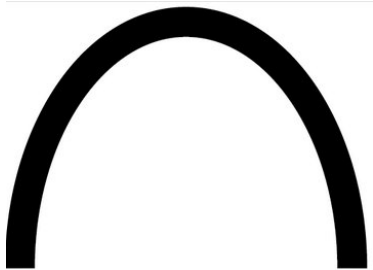
Raster Image



Vector Image

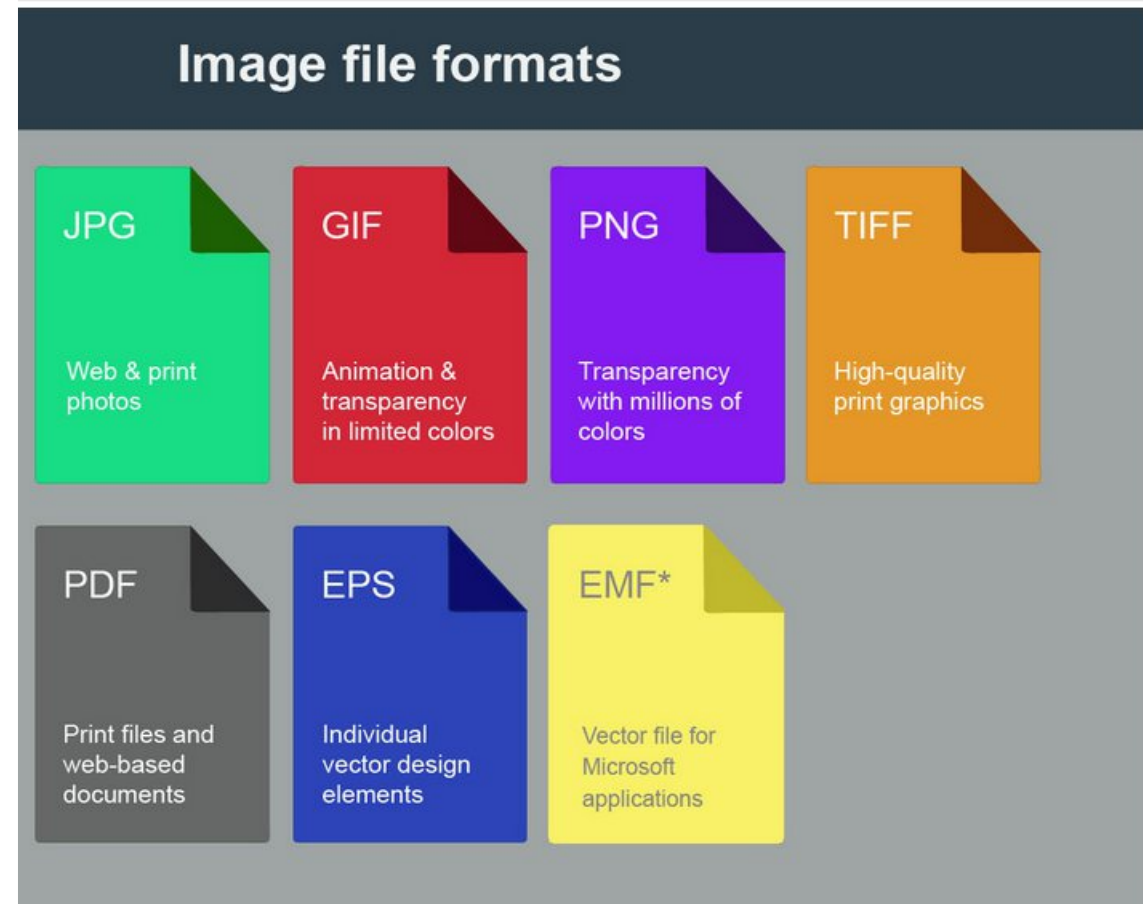


Raster Image



Vector Image

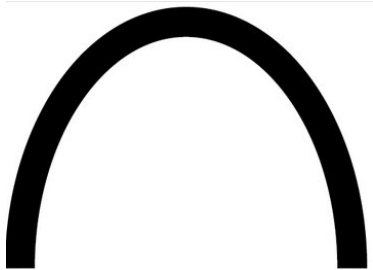
Beschreibend
z.B. Ellipse(M,F1,F2,S1,...,S4)



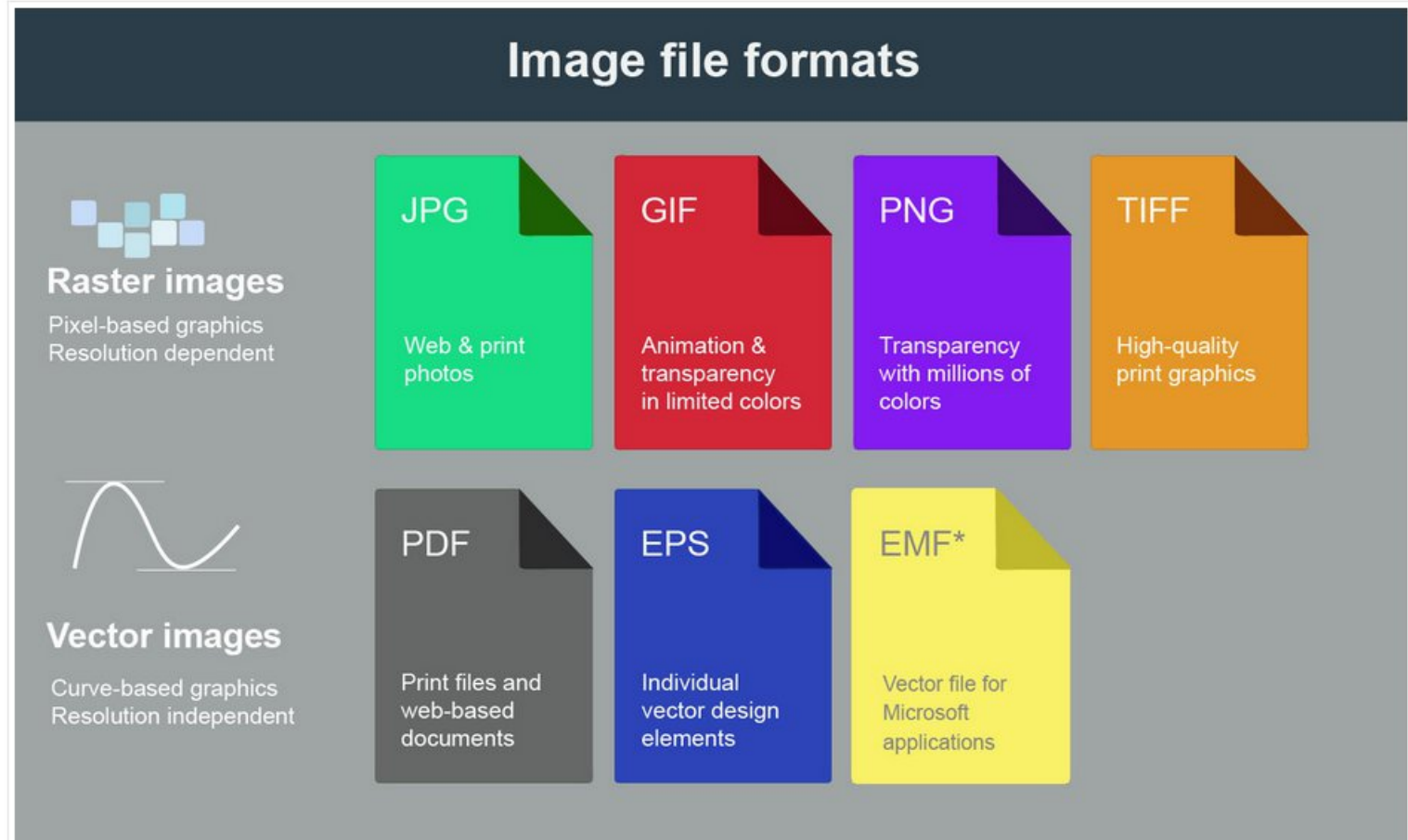
Raster- vs. Vectorimage



Raster Image



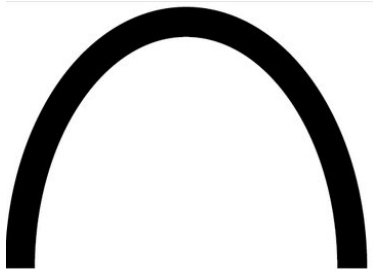
Vector Image



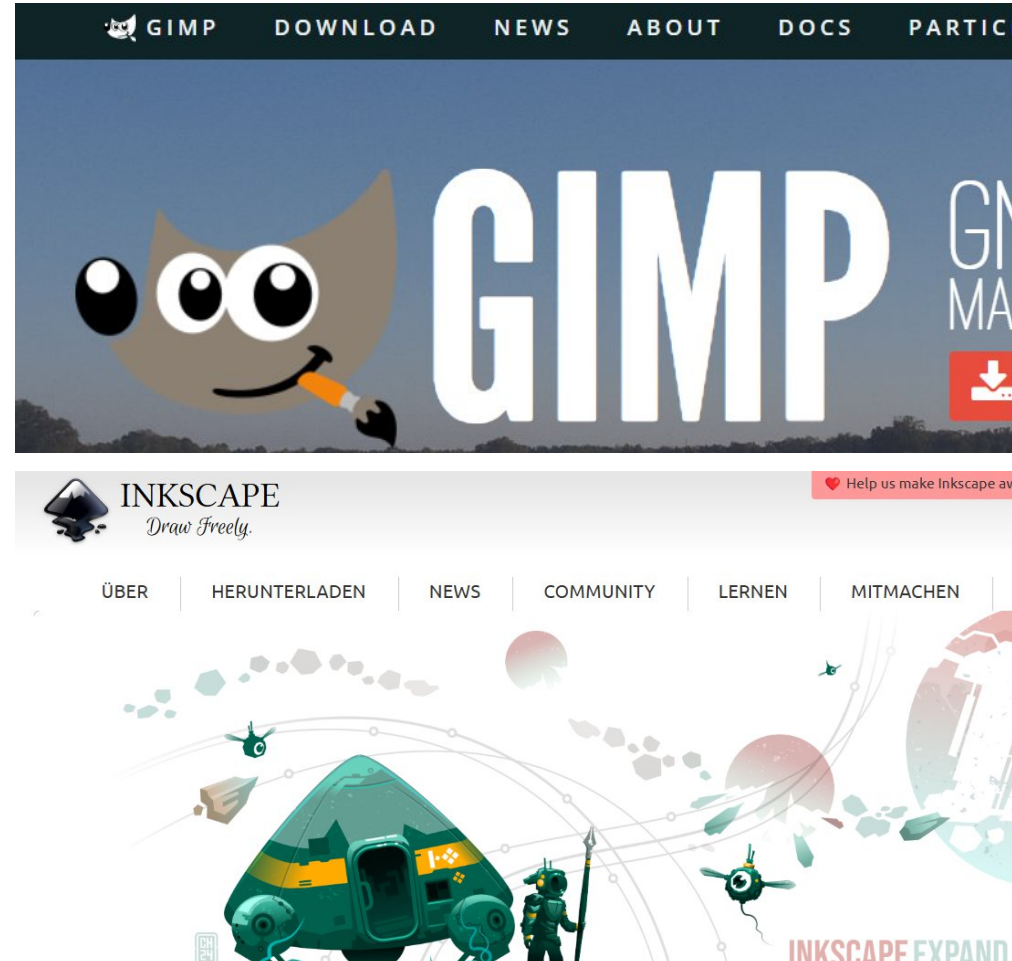
Raster- vs. Vectorimage: Grafikprogramme



Raster Image



Vector Image



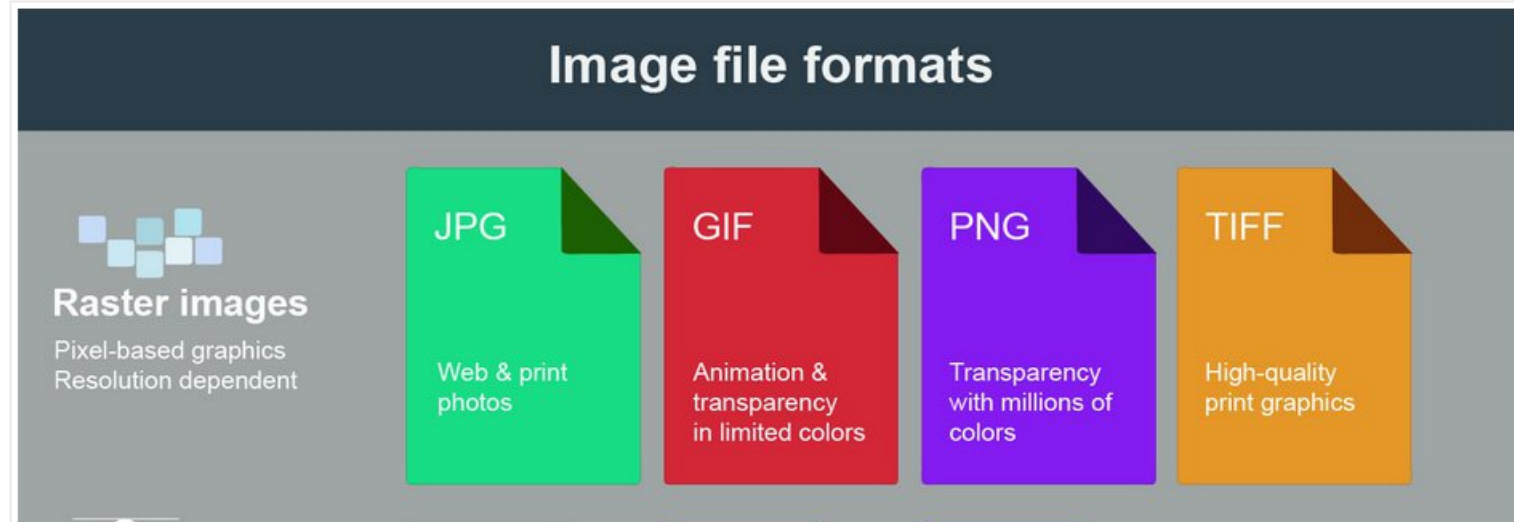
[GIMP](#)

[Inkscape](#)

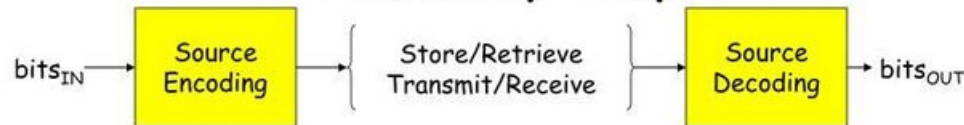
Verschiedene Kompressionsverfahren



Raster Image



Lossless vs. Lossy Compression



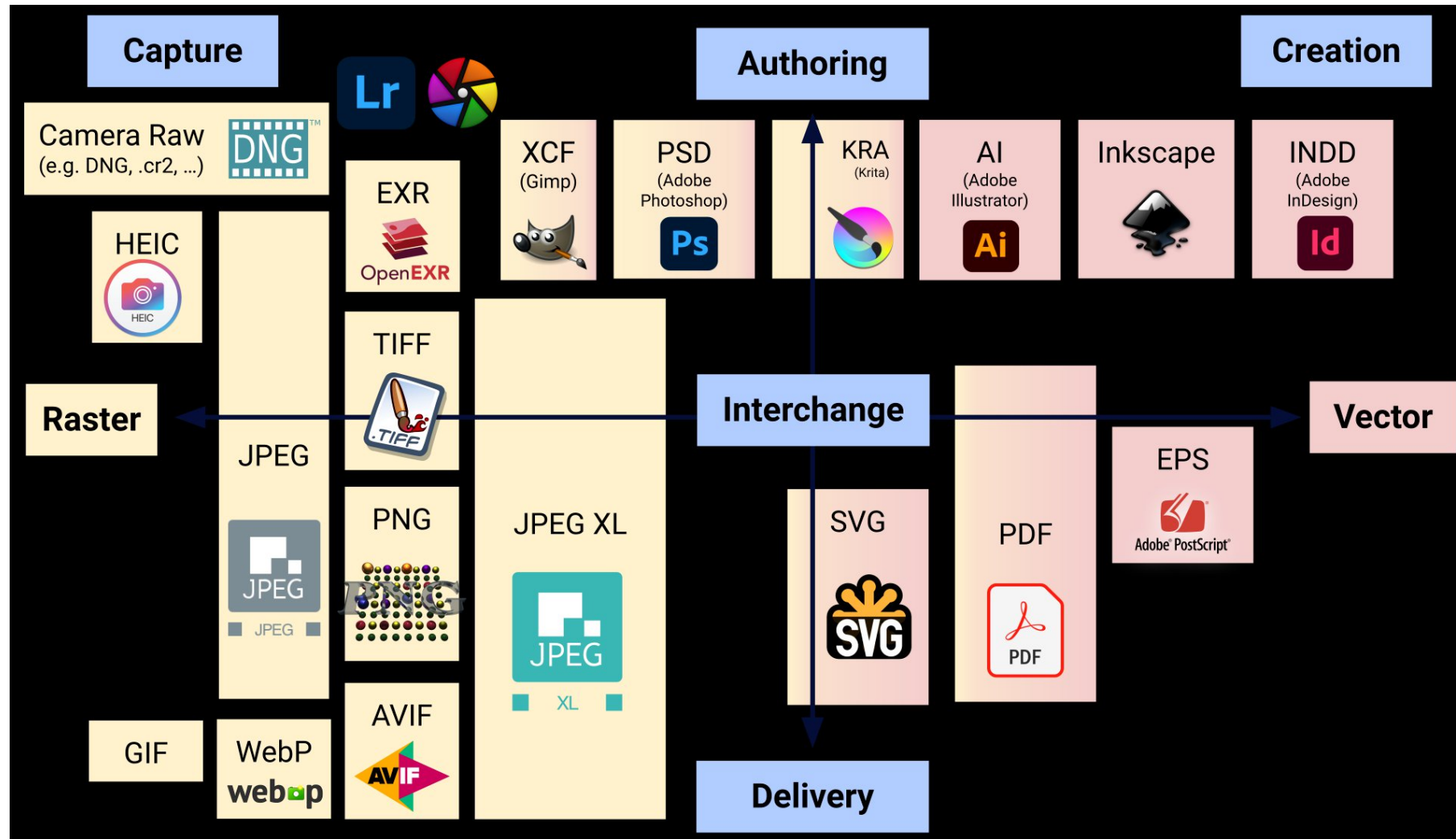
- For **lossless** data compression: $\text{bits}_{\text{OUT}} = \text{bits}_{\text{IN}}$
 - We can reconstruct the original bit stream exactly
 - $\text{bits}_{\text{OUT}} = \text{bits}_{\text{IN}}$
 - Usually used for "naturally digital" bit streams, e.g. documents, messages, datasets, ...
 - Examples: Huffman encoding, LZW, zip files, rar files

- For **lossy** encodings: $\text{bits}_{\text{OUT}} \approx \text{bits}_{\text{IN}}$
 - "Essential" information preserved
 - Appropriate for sampled data streams (audio, video) intended for human consumption via imperfect sensors (ears, eyes).

<https://www.barcode.graphics/barcode-image-file-formats/>

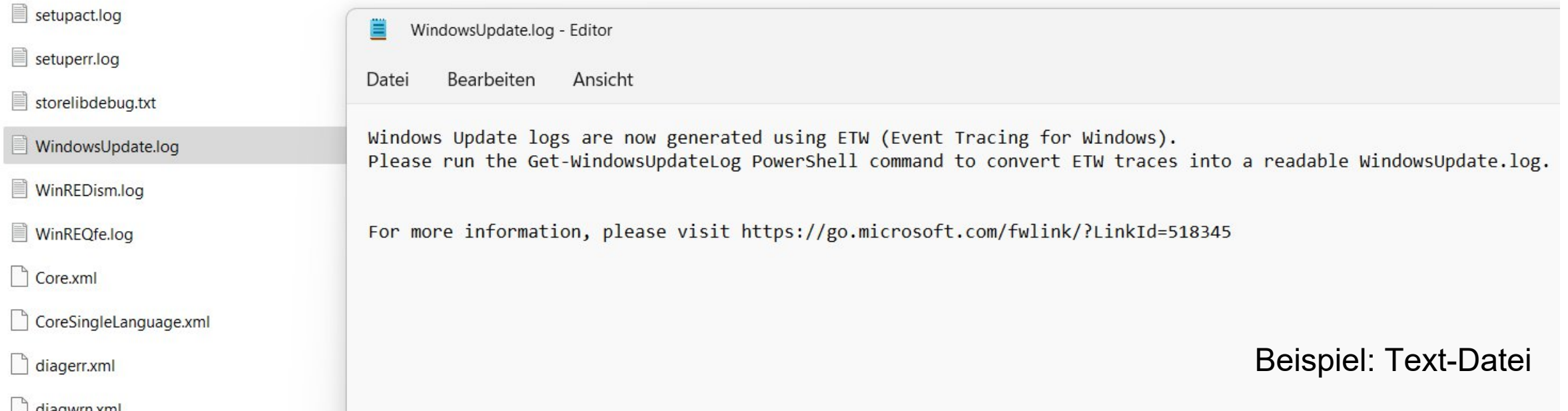
<https://in.pinterest.com/pin/lossy-compression-vs-lossless-compression--24136547997652905/>

Verschiedene Verwendungszwecke





- Text-Dateien: mit Standard-Editoren darstellbar und zu bearbeiten
- Binär-Dateien: Unformatiert, Byte für Byte abgespeichert, keine Zeilentrenner/Leerzeichen



Beispiel: Text-Datei



- Text-Dateien: mit Standard-Editoren darstellbar und zu bearbeiten
- Binär-Dateien: Unformatiert, Byte für Byte abgespeichert, keine Zeilentrenner/Leerzeichen
- **Vorteile**
 - Speicherung im Binärformat ist bei großen Datenmengen effizienter
 - Zugriff auf Binärdateien ist schneller (Lese-/Schreibgeschwindigkeit des Massenspeichers)
- **Nachteile**
 - Import in andere Programme (Tabellenkalkulation etc.) nicht direkt möglich
- Speicherbedarf einer `double` Variablen bei Abspeicherung (ohne Verlust an Genauigkeit und Wertebereich)

- Text-Dateien: mit Standard-Editoren darstellbar und zu bearbeiten
- Binär-Dateien: Unformatiert, Byte für Byte abgespeichert, keine Zeilentrenner/Leerzeichen
- **Vorteile**
 - Speicherung im Binärformat ist bei großen Datenmengen effizienter
 - Zugriff auf Binärdateien ist schneller (Lese-/Schreibgeschwindigkeit des Massenspeichers)
- **Nachteile**
 - Import in andere Programme (Tabellenkalkulation etc.) nicht direkt möglich
- Speicherbedarf einer `double` Variablen bei Abspeicherung (ohne Verlust an Genauigkeit und Wertebereich)

Binär	0000 0001 1000 1111 0010 0001 1100 1100 1011 1011 0111 1001 0010 1000 0101 0100	8 Byte
ASCII	1.12345678909876E308 (15 signifikante Stellen)	21 Byte

- wmpnetwk.exe
- wmpnetwk.mof
- wmpnss_color32.bmp**
- wmpnss_color32.jpg
- wmpnss_color48.bmp
- wmpnss_color48.jpg
- wmpnss_color48.png

[illegible]

Beispiel: Binär-Datei1

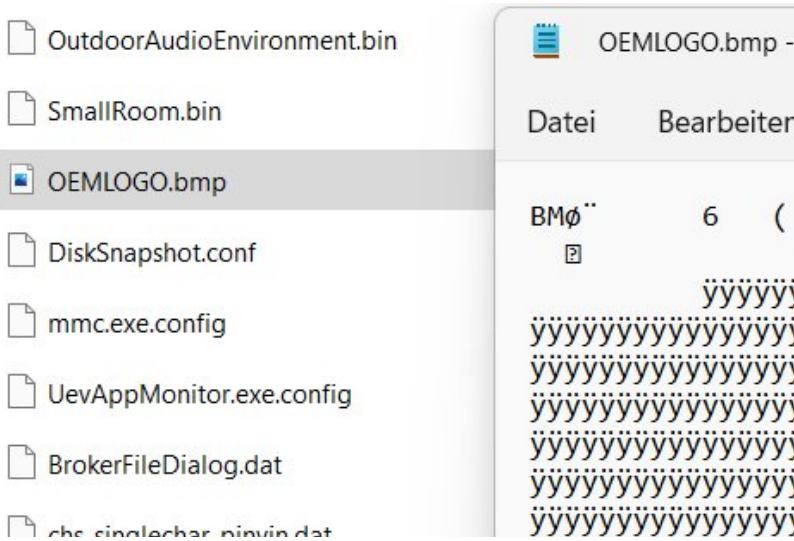
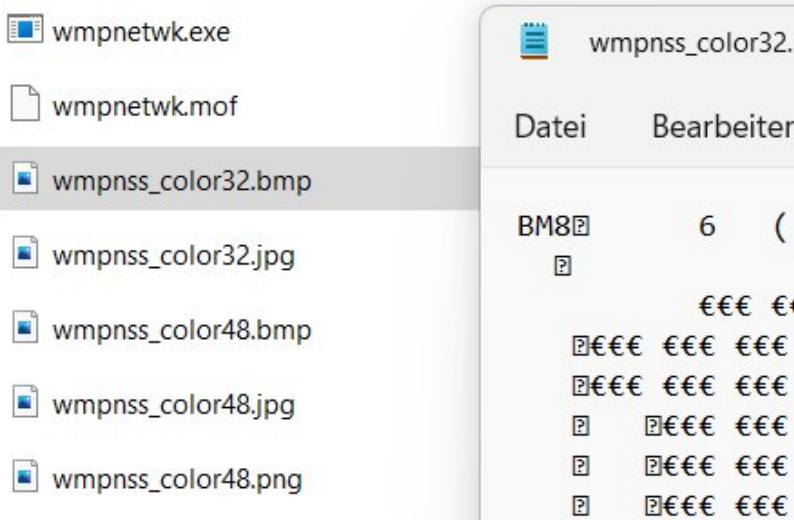
- OutdoorAudioEnvironment.bin
- SmallRoom.bin
- OEMLOGO.bmp**
- DiskSnapshot.conf
- mmc.exe.config
- UevAppMonitor.exe.config
- BrokerFileDialog.dat
- che_singlechar_nirvindat

$$BM\phi'' \quad 6 \quad (\quad x \quad x \quad ? \quad ? \quad \hat{A}'' \quad ?)$$
[illegible]

Beispiel: Binär-Datei2

KA

Dateiformat (Windows Bitmap; BMP)



Dateiformat (Version 3) [\[Bearbeiten | Quelltext bearbeiten \]](#)

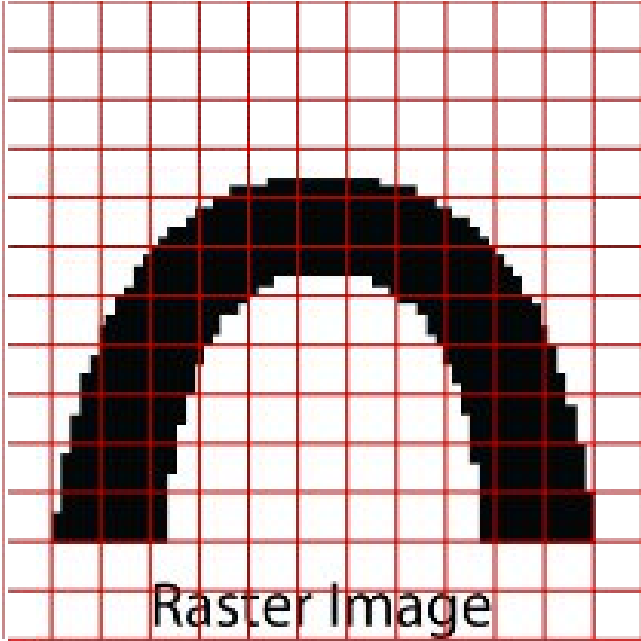
BMP-Dateien bestehen aus drei Teilen: dem Dateikopf, dem Informationsblock und den Bilddaten (siehe Schema rechts).

Im Folgenden bezeichnet WORD einen 16-Bit-vorzeichenlosen [Integer](#), DWORD einen 32-Bit-vorzeichenlosen Integer und LONG einen 32-Bit-kodierten Integer. BMP verwendet die [Little-Endian](#)-Konvention.

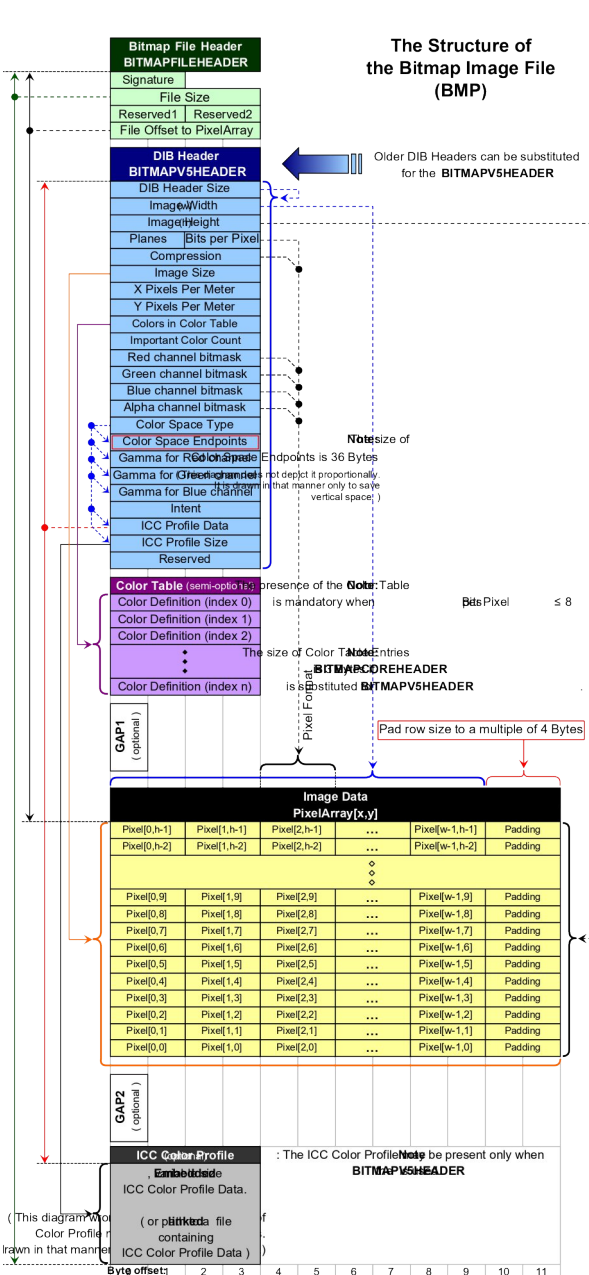
Dateikopf [\[Bearbeiten | Quelltext bearbeiten \]](#)

BITMAPFILEHEADER (Größe: 14 Byte)						
Offset (Byte)		Datentyp		Größe	Name	Inhalt
Dez	Hex	Windows-Style	C-Style			
0	0	WORD	uint16_t	2 Byte	bfType	ASCII-Zeichenkette "BM" (Hex: 0x42 0x4D, Dezimal: 66 77).
2	2	DWORD	uint32_t	4 Byte	bfSize	Größe der BMP-Datei in Byte. (unzuverlässig)
6	6	DWORD	uint32_t	4 Byte	bfReserved	Reserviert, von der Software abhängig, standardmäßig 0
10	A	DWORD	uint32_t	4 Byte	bfOffBits	Offset der Bilddaten in Byte vom Beginn der Datei an. Dieser hat nicht immer den Wert 54 (er ergibt sich aus 14 Byte Dateikopf und 40 Byte Informationsblock) und muss daher dynamisch ausgelesen werden, weil es sonst kein Bild kommt.

Dateiformat (Windows Bitmap; BMP)



Dateiformat (Windows Bitmap; BMP)



Dateikopf

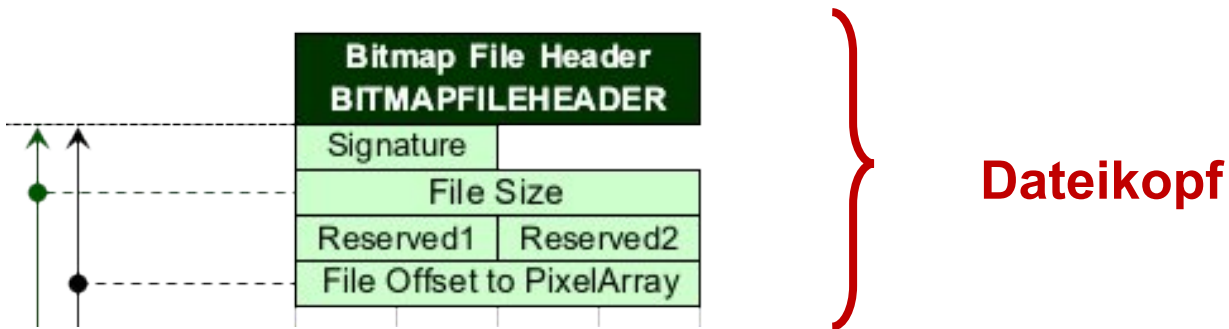
Informationsblock

Bilddaten

(optionales Farbprofil)



Dateiformat (Windows Bitmap; BMP) / Dateikopf

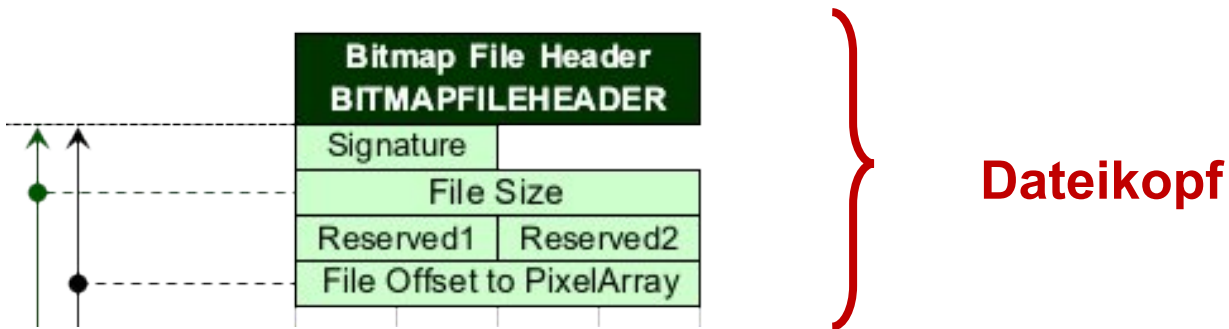


Dateikopf [[Bearbeiten](#) | [Quelltext bearbeiten](#)]

BITMAPFILEHEADER (Größe: 14 Byte)						
Offset (Byte)		Datentyp		Größe	Name	Inhalt
Dez	Hex	Windows-Style	C-Style			
0	0	WORD	uint16_t	2 Byte	bftype	ASCII-Zeichenkette "BM" (Hex: 0x42 0x4D, Dezimal: 66 77).
2	2	DWORD	uint32_t	4 Byte	bfSize	Größe der BMP-Datei in Byte. (unzuverlässig)
6	6	DWORD	uint32_t	4 Byte	bfReserved	Reserviert, von der Software abhängig, standardmäßig 0
10	A	DWORD	uint32_t	4 Byte	bfOffBits	Offset der Bilddaten in Byte vom Beginn der Datei an. Dieser hat nicht immer den Wert 54 (er ergibt sich aus 14 Byte Header + 40 Byte Infoblock) und muss daher dynamisch ausgelesen werden, weil es sonst ggf. zu Fehldarstellungen im Bild kommt.



Dateiformat (Windows Bitmap; BMP) / Dateikopf

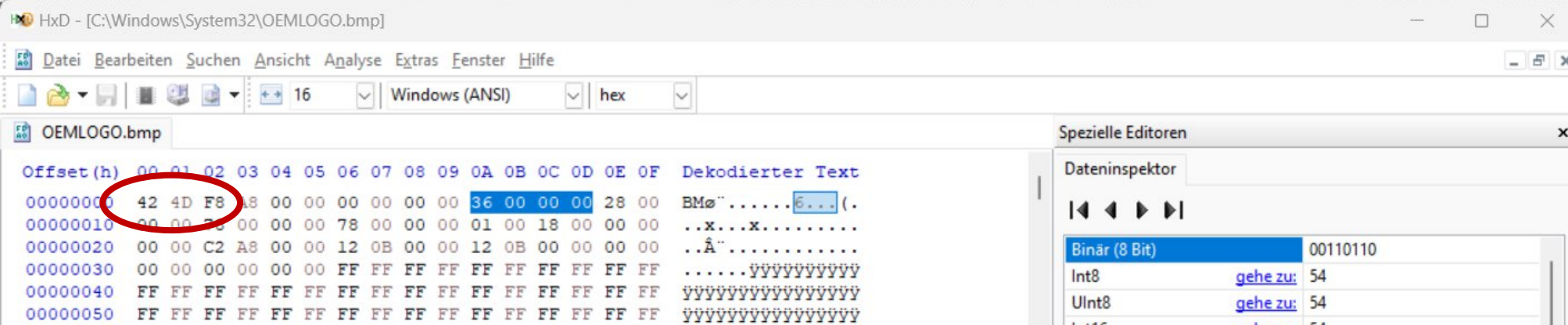


Dateikopf [Bearbeiten | Quelltext bearbeiten]

BITMAPFILEHEADER (Größe: 14 Byte)						
Offset (Byte)		Datentyp		Größe	Name	Inhalt
Dez	Hex	Windows-Style	C-Style			
0	0	WORD	uint16_t	2 Byte	bftype	ASCII-Zeichenkette "BM" (Hex: 0x42 0x4D, Dezimal: 66 77).
2	2	DWORD	uint32_t	4 Byte	bfSize	Größe der BMP-Datei in Byte. (unzuverlässig)
6	6	DWORD	uint32_t	4 Byte	bfReserved	Reserviert, von der Software abhängig, standardmäßig 0
10	A	DWORD	uint32_t	4 Byte	bfOffBits	Offset der Bilddaten in Byte vom Beginn der Datei an. Dieser hat nicht immer den Wert 54 (er ergibt sich aus 14 Byte Header + 40 Byte Infoblock) und muss daher dynamisch ausgelesen werden, weil es sonst ggf. zu Fehldarstellungen im Bild kommt.



Dateiformat (Windows Bitmap; BMP) / Dateikopf

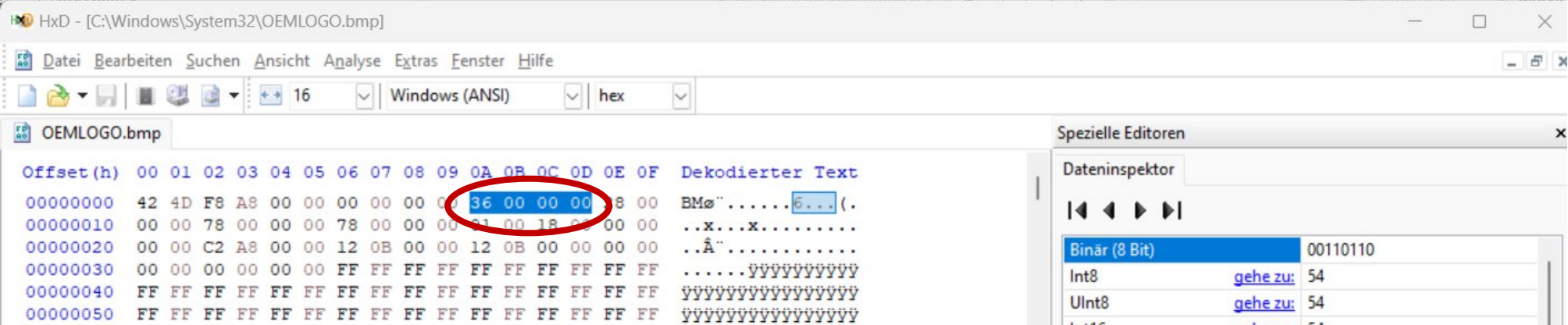


Dateikopf [Bearbeiten | Quelltext bearbeiten]

BITMAPFILEHEADER (Größe: 14 Byte)						
Offset (Byte)		Datentyp		Größe	Name	Inhalt
Dez	Hex	Windows-Style	C-Style			
0	0	WORD	uint16_t	2 Byte	bfType	ASCII-Zeichenkette "BM" (Hex: 0x42 0x4D, Dezimal: 66 77).
2	2	DWORD	uint32_t	4 Byte	bfSize	Größe der BMP-Datei in Byte. (unzuverlässig)
6	6	DWORD	uint32_t	4 Byte	bfReserved	Reserviert, von der Software abhängig, standardmäßig 0
10	A	DWORD	uint32_t	4 Byte	bfOffBits	Offset der Bilddaten in Byte vom Beginn der Datei an. Dieser hat nicht immer den Wert 54 (er ergibt sich aus 14 Byte Header + 40 Byte Infoblock) und muss daher dynamisch ausgelesen werden, weil es sonst ggf. zu Fehldarstellungen im Bild kommt.



Dateiformat (Windows Bitmap; BMP) / Dateikopf



Dateikopf [Bearbeiten | Quelltext bearbeiten]

BITMAPFILEHEADER (Größe: 14 Byte)						
Offset (Byte)		Datentyp		Größe	Name	Inhalt
Dez	Hex	Windows-Style	C-Style			
0	0	WORD	uint16_t	2 Byte	bfType	ASCII-Zeichenkette "BM" (Hex: 0x42 0x4D, Dezimal: 66 77).
2	2	DWORD	uint32_t	4 Byte	bfSize	Größe der BMP-Datei in Byte. (unzuverlässig)
6	6	DWORD	uint32_t	4 Byte	bfReserved	Reserviert, von der Software abhängig, standardmäßig 0
10	A	DWORD	uint32_t	4 Byte	bfOffBits	Offset der Bilddaten in Byte vom Beginn der Datei an. Dieser hat nicht immer den Wert 54 (er ergibt sich aus 14 Byte Header + 40 Byte Infoblock) und muss daher dynamisch ausgelesen werden, weil es sonst ggf. zu Fehldarstellungen im Bild kommt.



Dateiformat (Windows Bitmap; BMP) / Bilddaten



Image Data PixelFormat[x,y]					
Pixel[0,h-1]	Pixel[1,h-1]	Pixel[2,h-1]	...	Pixel[w-1,h-1]	Padding
Pixel[0,h-2]	Pixel[1,h-2]	Pixel[2,h-2]	...	Pixel[w-1,h-2]	Padding
◇ ◇ ◇					
Pixel[0,9]	Pixel[1,9]	Pixel[2,9]	...	Pixel[w-1,9]	Padding
Pixel[0,8]	Pixel[1,8]	Pixel[2,8]	...	Pixel[w-1,8]	Padding
Pixel[0,7]	Pixel[1,7]	Pixel[2,7]	...	Pixel[w-1,7]	Padding
Pixel[0,6]	Pixel[1,6]	Pixel[2,6]	...	Pixel[w-1,6]	Padding
Pixel[0,5]	Pixel[1,5]	Pixel[2,5]	...	Pixel[w-1,5]	Padding
Pixel[0,4]	Pixel[1,4]	Pixel[2,4]	...	Pixel[w-1,4]	Padding
Pixel[0,3]	Pixel[1,3]	Pixel[2,3]	...	Pixel[w-1,3]	Padding
Pixel[0,2]	Pixel[1,2]	Pixel[2,2]	...	Pixel[w-1,2]	Padding
Pixel[0,1]	Pixel[1,1]	Pixel[2,1]	...	Pixel[w-1,1]	Padding
Pixel[0,0]	Pixel[1,0]	Pixel[2,0]	...	Pixel[w-1,0]	Padding

Bilddaten

OEMLOGO.bmp

Offset (h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Dekodierter Text
00000000	42	4D	F8	A8	00	00	00	00	00	00	36	00	00	00	28	00	BMø.....6... (.
00000010	00	00	78	00	00	00	78	00	00	00	01	00	18	00	00	00	...x...x.....
00000020	00	00	C2	A8	00	00	12	0B	00	00	12	0B	00	00	00	00	..Â.....
00000030	00	00	00	00	00	00	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvv
00000040	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvv
00000050	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvv
00000060	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvv
00000070	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvv
00000080	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvv
00000090	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvv
000000A0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvv

Spezielle Editoren

Dateninspektor

Binär (8 Bit)

11111111

Int8	gehe zu:	-1
UInt8	gehe zu:	255
Int16	gehe zu:	Ungültig
UInt16	gehe zu:	Ungültig
Int24	gehe zu:	Ungültig
UInt24	gehe zu:	Ungültig



Bilddaten

„ bei BI_RGB [keine Kompression] ist die Länge jeder Zeile ein Vielfaches von 4 Bytes und wird, falls erforderlich, mit Nullbytes aufgefüllt.“

https://de.wikipedia.org/wiki/Windows_Bitmap

<https://solarianprogrammer.com/2018/11/19/cpp-reading-writing-bmp-images/>

[illegible]

Dateiformat (Windows Bitmap; BMP) / Bilddaten



Image Data PixelFormat[x,y]					
Pixel[0,h-1]	Pixel[1,h-1]	Pixel[2,h-1]	...	Pixel[w-1,h-1]	Padding
Pixel[0,h-2]	Pixel[1,h-2]	Pixel[2,h-2]	...	Pixel[w-1,h-2]	Padding
◇ ◇ ◇					
Pixel[0,9]	Pixel[1,9]	Pixel[2,9]	...	Pixel[w-1,9]	Padding
Pixel[0,8]	Pixel[1,8]	Pixel[2,8]	...	Pixel[w-1,8]	Padding
Pixel[0,7]	Pixel[1,7]	Pixel[2,7]	...	Pixel[w-1,7]	Padding
Pixel[0,6]	Pixel[1,6]	Pixel[2,6]	...	Pixel[w-1,6]	Padding
Pixel[0,5]	Pixel[1,5]	Pixel[2,5]	...	Pixel[w-1,5]	Padding
Pixel[0,4]	Pixel[1,4]	Pixel[2,4]	...	Pixel[w-1,4]	Padding
Pixel[0,3]	Pixel[1,3]	Pixel[2,3]	...	Pixel[w-1,3]	Padding
Pixel[0,2]	Pixel[1,2]	Pixel[2,2]	...	Pixel[w-1,2]	Padding
Pixel[0,1]	Pixel[1,1]	Pixel[2,1]	...	Pixel[w-1,1]	Padding
Pixel[0,0]	Pixel[1,0]	Pixel[2,0]	...	Pixel[w-1,0]	Padding

Wie groß ist x und y?

Bilddaten

OEMLOGO.bmp

Offset (h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Dekodierter Text
00000000	42	4D	F8	A8	00	00	00	00	00	00	36	00	00	00	28	00	BMø.....6... (.
00000010	00	00	78	00	00	00	78	00	00	00	01	00	18	00	00	00	...x...x.....
00000020	00	00	C2	A8	00	00	12	0B	00	00	12	0B	00	00	00	00	..Â.....
00000030	00	00	00	00	00	00	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	[yvvvvvvvvv
00000040	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvvv
00000050	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvvv
00000060	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvvv
00000070	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvvv
00000080	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvvv
00000090	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvvv
000000A0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	vvvvvvvvvvvvvvvvv

Spezielle Editoren

Dateninspektor

Binär (8 Bit)

11111111

Int8	gehe zu:	-1
UInt8	gehe zu:	255
Int16	gehe zu:	Ungültig
UInt16	gehe zu:	Ungültig
Int24	gehe zu:	Ungültig
UInt24	gehe zu:	Ungültig



Dateiformat (Windows Bitmap; BMP) / Bilddaten



Image Data PixelFormat[x,y]					
Pixel[0,h-1]	Pixel[1,h-1]	Pixel[2,h-1]	...	Pixel[w-1,h-1]	Padding
Pixel[0,h-2]	Pixel[1,h-2]	Pixel[2,h-2]	...	Pixel[w-1,h-2]	Padding
⋮					
Pixel[0,9]	Pixel[1,9]	Pixel[2,9]	...	Pixel[w-1,9]	Padding
Pixel[0,8]	Pixel[1,8]	Pixel[2,8]	...	Pixel[w-1,8]	Padding
Pixel[0,7]	Pixel[1,7]	Pixel[2,7]	...	Pixel[w-1,7]	Padding
Pixel[0,6]	Pixel[1,6]	Pixel[2,6]	...	Pixel[w-1,6]	Padding
Pixel[0,5]	Pixel[1,5]	Pixel[2,5]	...	Pixel[w-1,5]	Padding
Pixel[0,4]	Pixel[1,4]	Pixel[2,4]	...	Pixel[w-1,4]	Padding
Pixel[0,3]	Pixel[1,3]	Pixel[2,3]	...	Pixel[w-1,3]	Padding
Pixel[0,2]	Pixel[1,2]	Pixel[2,2]	...	Pixel[w-1,2]	Padding
Pixel[0,1]	Pixel[1,1]	Pixel[2,1]	...	Pixel[w-1,1]	Padding
Pixel[0,0]	Pixel[1,0]	Pixel[2,0]	...	Pixel[w-1,0]	Padding

Bilddaten

Was bedeutet hier der Wert 255?
Wird 1 Byte oder mehr zum kodieren verwendet?

Offset (h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Dekodierter Text
00000000	42	4D	F8	A8	00	00	00	00	00	00	36	00	00	00	28	00	BMø.....6... (.
00000010	00	00	78	00	00	00	78	00	00	00	01	00	00	00	00	00	...x...x.....
00000020	00	00	C2	A8	00	00	12	00	00	00	12	0B	00	00	00	00	..Â.....
00000030	00	00	00	00	00	00	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	[y]yyyyyyyy
00000040	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyy
00000050	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyy
00000060	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyy
00000070	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyy
00000080	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyy
00000090	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyy
000000A0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyy

Spezielle Editoren		
Dateninspektor		
◀◀▶▶		
Binär (8 Bit)		11111111
Int8	gehe zu:	-1
UInt8	gehe zu:	255
Int16	gehe zu:	Ungültig
UInt16	gehe zu:	Ungültig
Int24	gehe zu:	Ungültig
UInt24	gehe zu:	Ungültig

K
A

Dateiformat (Windows Bitmap; BMP) / Informationsblock



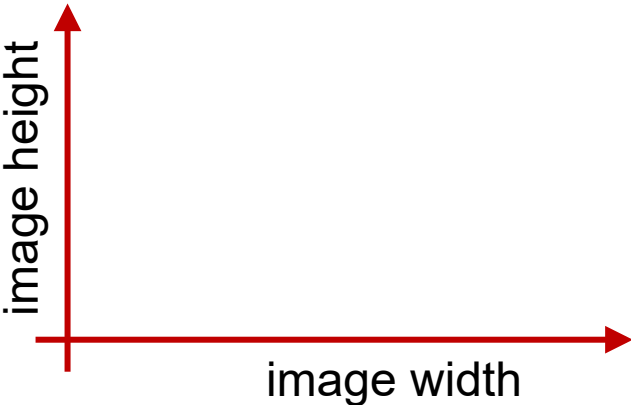
DIB Header BITMAPV5HEADER	
DIB Header Size	
Image Width (w)	
Image Height (h)	
Planes	Bits per Pixel
Compression	
Image Size	
X Pixels Per Meter	
Y Pixels Per Meter	
Colors in Color Table	
Important Color Count	
Red channel bitmask	
Green channel bitmask	
Blue channel bitmask	
Alpha channel bitmask	
Color Space Type	
Color Space Endpoints	
Gamma for Red channel	
Gamma for Green channel	
Gamma for Blue channel	
Intent	
ICC Profile Data	
ICC Profile Size	
Reserved	
Color Table (semi-optional)	
Color Definition (index 0)	
Color Definition (index 1)	
Color Definition (index 2)	
⋮	
Color Definition (index n)	

Informationsblock

Dateiformat (Windows Bitmap; BMP) / Größeninformationen



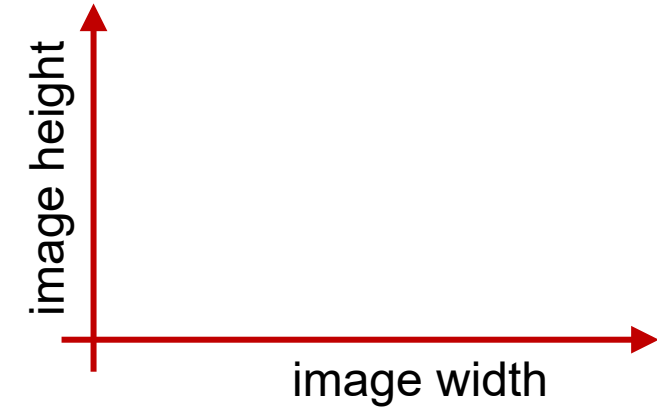
DIB Header BITMAPV5HEADER	
DIB Header Size	
Image Width (w)	
Image Height (h)	
Planes	Bits per Pixel
Compression	
Image Size	
X Pixels Per Meter	
Y Pixels Per Meter	
Colors in Color Table	
Important Color Count	
Red channel bitmask	
Green channel bitmask	
Blue channel bitmask	
Alpha channel bitmask	
Color Space Type	
Color Space Endpoints	
Gamma for Red channel	
Gamma for Green channel	
Gamma for Blue channel	
Intent	
ICC Profile Data	
ICC Profile Size	
Reserved	
Color Table (semi-optional)	
Color Definition (index 0)	
Color Definition (index 1)	
Color Definition (index 2)	
⋮	
Color Definition (index n)	



Informationsblock

Offset (hex)	Offset (dec)	Size (bytes)	Windows BITMAPINFOHEADER ^[2]
0E	14	4	the size of this header, in bytes (40)
12	18	4	the bitmap width in pixels (signed integer)
16	22	4	the bitmap height in pixels (signed integer)
1A	26	2	the number of color planes (must be 1)
1C	28	2	the number of bits per pixel, which is the color depth of the image. Typical values are 1, 4, 8, 16, 24 and 32.
1E	30	4	the compression method being used. See the next table for a list of possible values

Image Width (w)
Image Height (h)



}

C
C
C
C
C

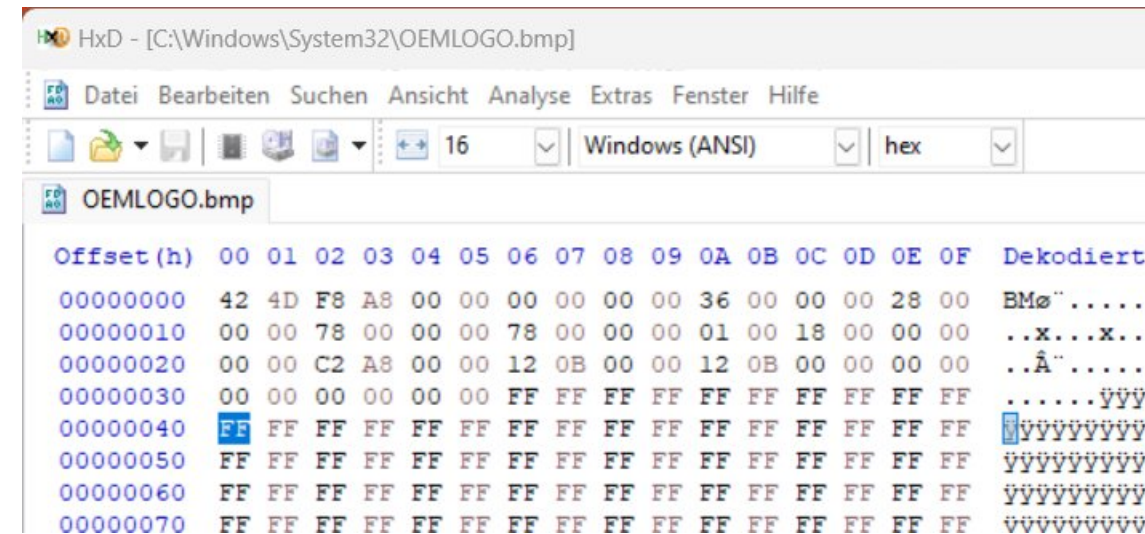
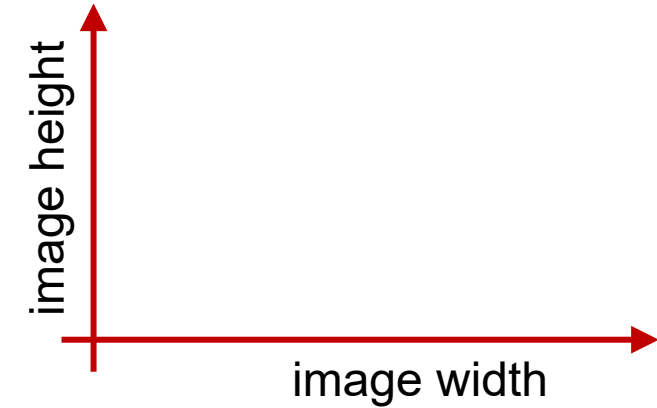
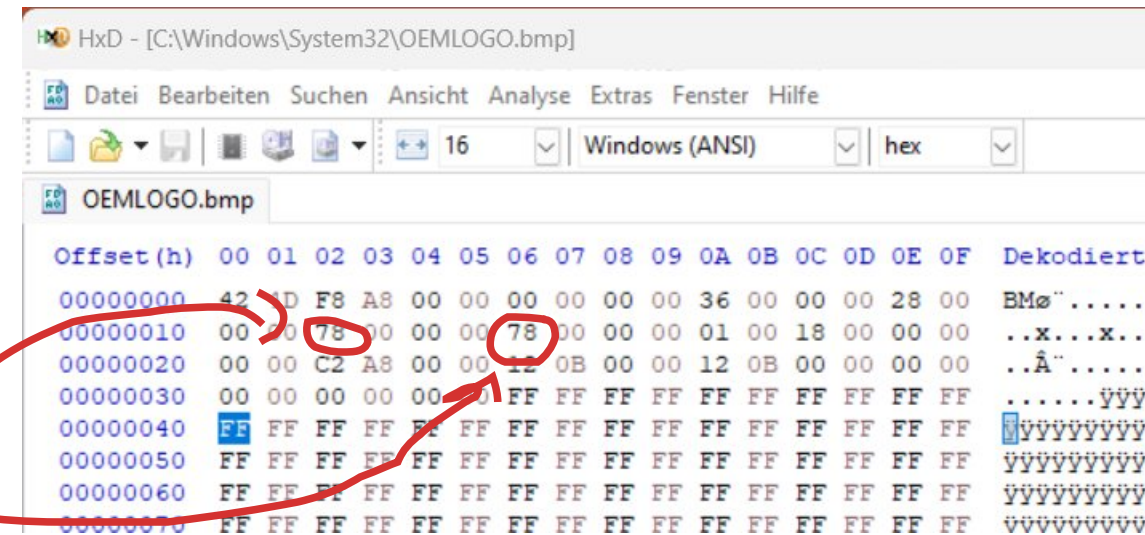


Image Width (w)
Image Height (h)



}

C
C
C
C
C



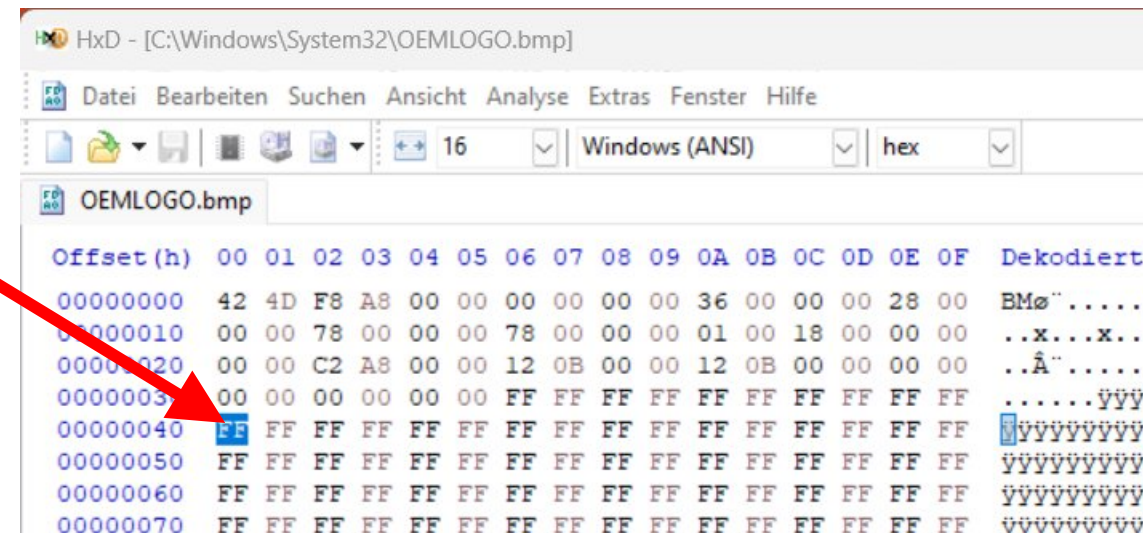
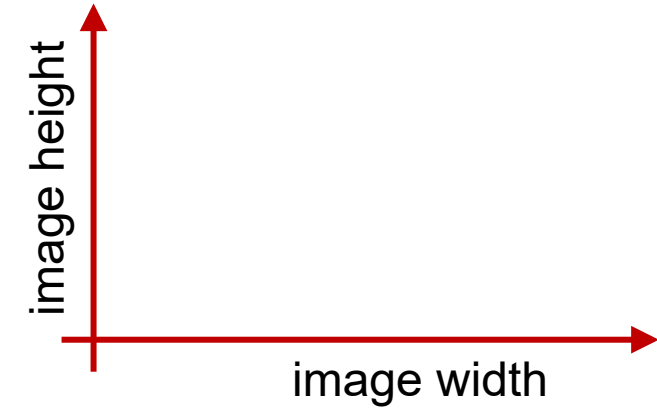
Offset (hex)	Offset (dec)	Size (bytes)	Windows 3.11
0E	14	4	the size of this header, in bytes (40)
12	18	4	the bitmap width in pixels (signed integer)
16	22	4	the bitmap height in pixels (signed integer)
1A	26	2	the number of color planes (must be 1)
1C	28	2	the number of bits per pixel, which is the number of bits per color plane. Can be 1, 2, 4, 8, 16, 24 and 32.
1E	30	4	the compression method being used. See the "Compression" section.

[illegible]

Offset (hex)	Offset (dec)	Size (bytes)	Windows E
0E	14	4	the size of this header, in bytes (40)
12	18	4	the bitmap width in pixels (signed integer)
16	22	4	the bitmap height in pixels (signed integer)
1A	26	2	the number of color planes (must be 1)
1C	28	2	the number of bits per pixel, which is the 8, 16, 24 and 32
1E	30	4	the compression method being used. See

[illegible]

8 Bits per Pixel aber was bedeutet hier der Wert 255?

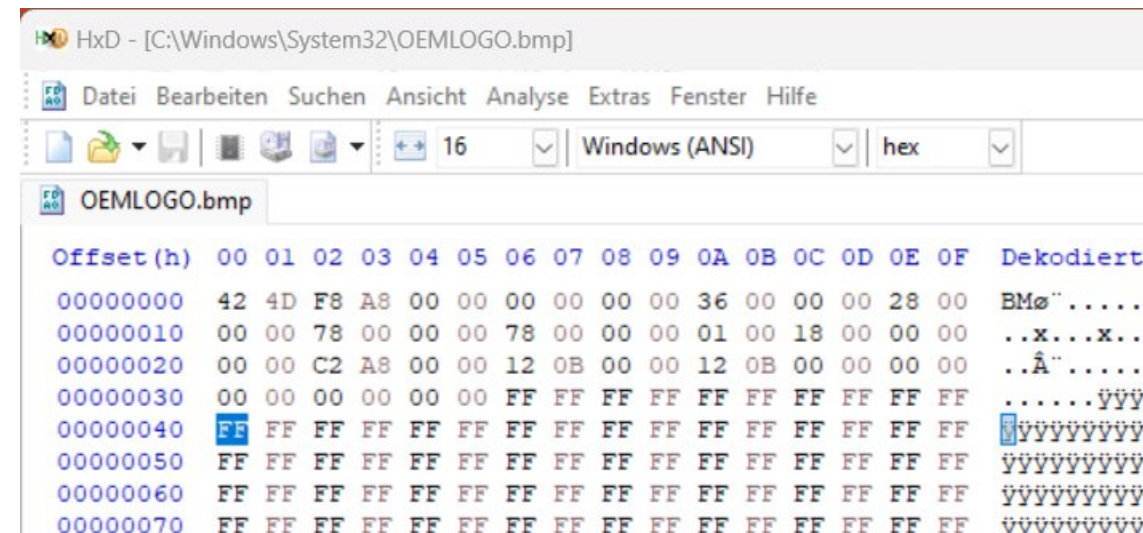


Offset (hex)	Offset (dec)	Size (bytes)	Windows E
2E	46	4	the number of colors in the color palette, or 0 to default to 2^n

A 2D coordinate system with a vertical axis labeled 'image height' and a horizontal axis labeled 'image width'. The axes are represented by red arrows originating from a common point. The vertical axis points upwards, and the horizontal axis points to the right. The labels are in a black, sans-serif font, positioned near their respective axes.

}

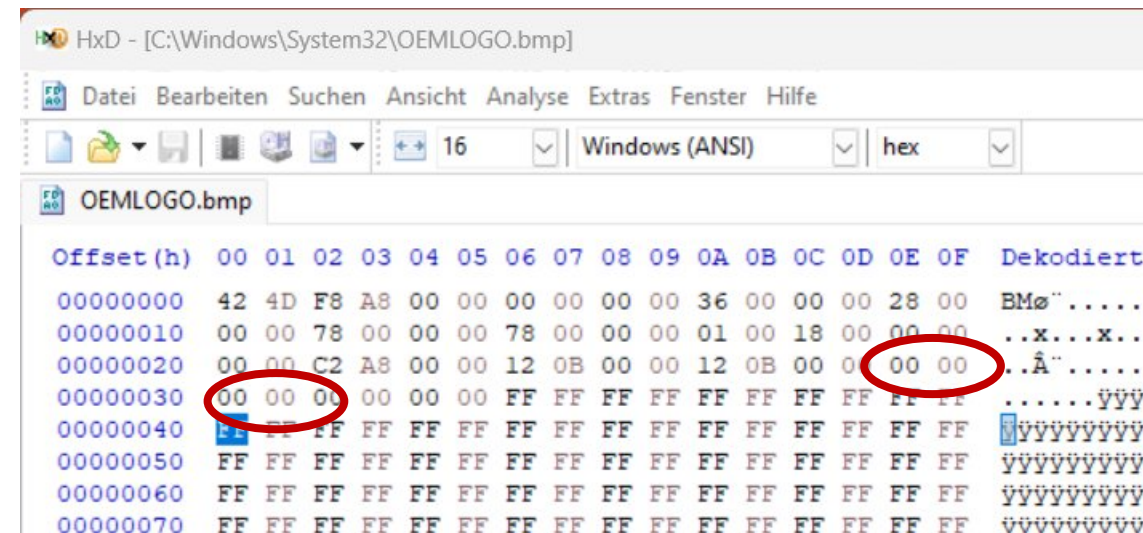
each entry in the color table occupies
4 bytes, in the order blue, green, red, 0x00



Offset (hex)	Offset (dec)	Size (bytes)	Windows E
2E	46	4	the number of colors in the color palette, or 0 to default to 2^n

A 2D coordinate system with a vertical axis labeled 'image height' and a horizontal axis labeled 'image width'. The axes are represented by red arrows originating from a common point. The vertical axis points upwards and is labeled 'image height' in black text. The horizontal axis points to the right and is labeled 'image width' in black text.

each entry in the color table occupies
4 bytes, in the order blue, green, red, 0x00





C++

Wiederholung implicite/explicite cast



- **Implizite Typenumwandlung:** Standard Typenumwandlungen für eingebaute Typen. Die Typenumwandlung wird nicht ausdrücklich hingeschrieben & der Compiler gibt keine Fehlermeldung.
 - char, signed char, unsigned char, short int oder unsigned short int kann ohne Informationsverlust in einen int-Wert umgewandelt werden (englisch integral promotion).
 - Integrale Typen sind untereinander konvertierbar. Im C++-Standard werden alle Fälle, die nicht zur obigen »integral promotion« gehören, integral conversion genannt. Sonderfälle sind die Typen bool und enum
 - float kann in einen double- oder einen Integer-Wert umgewandelt werden und umgekehrt.
 - Für bool gilt ähnlich wie im int-Fall: true wird 1.0, false wird 0.0, 0.0 wird false und ein float- bzw. double-Ausdruck ungleich 0 wird true.



Wiederholung implicite/explicite cast



- Implizite Typenumwandlung: Standard Typenumwandlungen für eingebaute Typen. Die Typenumwandlung wird nicht ausdrücklich hingeschrieben & der Compiler gibt keine Fehlermeldung.
 - char, signed char, unsigned char, short int oder unsigned short int kann ohne Informationsverlust in einen int-Wert umgewandelt werden (englisch integral promotion).
 - Integrale Typen sind untereinander konvertierbar. Im C++-Standard werden alle Fälle, die nicht zur obigen »integral promotion« gehören, integral conversion genannt. Sonderfälle sind die Typen bool und enum
 - float kann in einen double- oder einen Integer-Wert umgewandelt werden und umgekehrt.
 - Für bool gilt ähnlich wie im int-Fall: true wird 1.0, false wird 0.0, 0.0 wird false und ein float- bzw. double-Ausdruck ungleich 0 wird true.
- **Explizite Typenumwandlung:** Benutzen des **static_cast**-Operators

```
char c {'x'};  
int i = static_cast<int>(c);           // Typumwandlung char → int
```

bedeutet, dass der Wert der Variablen i eine int-Repräsentation der char-Variablen c ist. Andere, einfachere Schreibweisen sind ebenfalls möglich:

```
i = int(c);                           // oder
```

```
i = (int) c;
```



reinterpret_cast-Operator



Explizite Typenumwandlung:

- Benutzen des **static_cast**-Operators

```
char c {'x'};  
int i = static_cast<int>(c);           // Typumwandlung char → int
```

bedeutet, dass der Wert der Variablen `i` eine `int`-Repräsentation der `char`-Variablen `c` ist. Andere, einfachere Schreibweisen sind ebenfalls möglich:

```
i = int(c);                           // oder
```

```
i = (int) c;
```

überprüft Typenverträglichkeit!

```
#include <iostream>  
using namespace std;  
  
int main(){  
    int a = 10;  
    double* ptr_double_a = nullptr;  
  
    ptr_double_a = &a;  
  
    return 0;  
}
```



reinterpret_cast-Operator



Explizite Typenumwandlung:

- Benutzen des **static_cast**-Operators

```
char c {'x'};  
int i = static_cast<int>(c);           // Typumwandlung char → int
```

bedeutet, dass der Wert der Variablen `i` eine `int`-Repräsentation der `char`-Variablen `c` ist. Andere, einfachere Schreibweisen sind ebenfalls möglich:

```
i = int(c);                           // oder
```

```
i = (int) c;
```

überprüft Typenverträglichkeit!

- Benutzen des **reinterpret_cast**-Operators

Verzichtet weitgehend **auf Prüfung** der Typenverträglichkeit!

Ermöglicht, dass ein beliebiger Zeiger in einen anderen Zeigertyp konvertiert wird.

Ermöglicht es außerdem, einen beliebigen ganzzahligen Typ in einen beliebigen Zeigertyp und umgekehrt zu konvertieren.

```
#include <iostream>  
using namespace std;  
  
int main(){  
    int a = 10;  
    double* ptr_double_a = nullptr;  
  
    ptr_double_a = reinterpret_cast<double*>(&a);  
    cout << *ptr_int_a << endl;  
  
    return 0;  
}
```

Wiederholung FileIO – Text-Datei



Bisher wurden reine Textdateien betrachtet:

- Die Ein- und Ausgabeoperatoren >> und << sind bei Dateien ebenso wie bei der Standardein- und -ausgabe verwendbar.
 - Der Header **<fstream>** enthält die vom Compiler verlangten Beschreibungen für Dateiobjekte.
1. Es wird ein **Dateiobjekt** mit einem beliebigen **Namen** definiert, das von nun an im Programm verwendet wird. Der Datentyp des Dateiobjekts ist ifstream für Ein- und ofstream für Ausgabedateien (Abkürzungen für input file stream beziehungsweise output file stream).
 2. Die Verbindung des Dateiobjekts zu einer existierenden oder anzulegenden Datei auf Betriebssystemebene wird mit der Funktion **open()** hergestellt, d.h. die **Datei wird »geöffnet«**.

```
ifstream datei;  
datei.open("dateiname.txt");  
// ist dasselbe wie  
ifstream datei("dateiname.txt");
```

3. Die Verbindung wird mit der **Funktion close()** wieder gelöst; man sagt auch, dass die **Datei geschlossen** wird. Ab diesem Zeitpunkt kann die Datei wieder von anderen Benutzern oder Programmen benutzt werden.

Wiederholung FileIO – Text-Datei / Beispiel: Speichern in Datei



Der Header **<fstream>** enthält die vom Compiler verlangten Beschreibungen für Dateiobjekte.

1. Es wird ein **Dateiobjekt** mit dem **Namen** „Ausgabedatei“ definiert.
Der Datentyp des Dateiobjekts ist ofstream für Ausgabedatei.
(Abkürzungen für output file stream).
2. Die Verbindung des Dateiobjekts zu einer existierenden oder anzulegenden Datei auf Betriebssystemebene wird mit der Funktion **open()** hergestellt, d.h. die **Datei wird »geöffnet«**.
 - a. Mit der Funktion **good()** wird geprüft, ob ein Fehler vorliegt (z.B. Dateiende, Datei kann nicht geöffnet werden, r/w Fehler)
 - b. Mit dem Doppelpfeil << wird wie bei cout auf die Ausgabe „geschrieben“
3. Die **Funktion close()** schließt die Datei.

```
#include <fstream>
#include <string>
int main() {
    std::ofstream Ausgabedatei;
    Ausgabedatei.open("test.txt");
    if (Ausgabedatei.good()) {
        Ausgabedatei << "Spass mit File I/O" << std::endl;
    }
    Ausgabedatei.close();
    return 0;
}
```

Wiederholung FileIO – Text-Datei / Beispiel: Lesen aus Datei



Der Header **<fstream>** enthält die vom Compiler verlangten Beschreibungen für Dateiobjekte.

1. Es wird ein **Dateiobjekt** mit dem **Namen** „Eingabedatei“ definiert.
Der Datentyp des Dateiobjekts ist **ifstream** für Eingabedatei.
(Abkürzungen für input file stream).
2. Die Verbindung des Dateiobjekts zu einer existierenden oder anzulegenden Datei auf Betriebssystemebene wird mit der Funktion **open()** hergestellt, d.h. die **Datei wird »geöffnet«**.
 - a. Mit der Funktion **good()** wird geprüft, ob ein Fehler vorliegt (z.B. **Dateiende**, Datei kann nicht geöffnet werden, r/w Fehler)
 - b. Mit der Funktion **getline()** wird eine Zeile aus der Datei gelesen. Als Argumente werden der Name des Dateiobjektes und ein String übergeben (in welchen die Informationen gespeichert werden).
3. Die **Funktion close()** schließt die Datei.

```
#include <fstream>
#include <string>
int main() {
    std::ifstream Eingabedatei;
    Eingabedatei.open("test.txt");
    if (Eingabedatei.good()) {
        std::string zeile;
        std::getline(Eingabedatei, zeile);
        std::cout << zeile << std::endl;
    }
    Eingabedatei.close();
    return 0;
}
```

Wiederholung FileIO – Text-Datei (Beispiel 1)



```
#include <iostream>
#include <fstream>
#include <string>
int main() {
    std::ofstream Ausgabedatei("test.txt");
    if (Ausgabedatei.good()) {
        Ausgabedatei << "Spass mit File I/O" << std::endl;
    } else {
        std::cout << "Datei konnte nicht geöffnet werden" << std::endl;
    }
    Ausgabedatei.close();
    return 0;
}
```

```
PS C:\EigeneDaten\Vorlesungen_2023_WS\Informatik1\FirstPrograms> cat .\test.txt
Spass mit File I/O
```



Wiederholung FileIO – Text-Datei (Beispiel 2)



```
#include <iostream>
#include <fstream>
#include <string>
int main() {
    std::ifstream Eingabedatei("test.txt");
    if (!Eingabedatei.good()) {std::cout << "Datei konnte nicht geöffnet werden" << std::endl;}
    while (Eingabedatei.good()) {
        std::string zeile;
        std::getline(Eingabedatei, zeile);
        std::cout << zeile;
    }
    return 0;
}
```

```
PS C:\EigeneDaten\Vorlesungen_2023_WS\Informatik1\FirstPrograms> .\FileIO.exe
Spass mit File I/O
PS C:\EigeneDaten\Vorlesungen_2023_WS\Informatik1\FirstPrograms> █
```



- Text-Dateien: mit Standard-Editoren darstellbar und zu bearbeiten
- Binär-Dateien: Unformatiert, Byte für Byte abgespeichert, keine Zeilentrenner/Leerzeichen
- **Vorteile**
 - Speicherung im Binärformat ist bei großen Datenmengen effizienter
 - Zugriff auf Binärdateien ist schneller (Lese-/Schreibgeschwindigkeit des Massenspeichers)
- **Nachteile**
 - Import in andere Programme (Tabellenkalkulation etc.) nicht direkt möglich
- Speicherbedarf einer `double` Variablen bei Abspeicherung (ohne Verlust an Genauigkeit und Wertebereich)

Binär	0000 0001 1000 1111 0010 0001 1100 1100 1011 1011 0111 1001 0010 1000 0101 0100	8 Byte
ASCII	1.12345678909876E308 (15 signifikante Stellen)	21 Byte

Binary File I/O



- Der binäre Datentransfer erfolgt unformatiert, d.h. die Daten werden in der *internen* Darstellung direkt geschrieben bzw. gelesen.

Beispiel: eine Integer-Zahl (==1) wird nicht als ASCII-Ziffernzeichen formatiert abgespeichert sondern die Bitrepräsentation:

```
02.02.2024 09:27 4 test_int.dat

test_int.dat
Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Dekodierter Text
00000000 01 00 00 00 ....
```

Spezielle Editoren

Dateninspektor

Navigation icons

Binär (8 Bit)	00000001
Int8	gehe zu: 1
UInt8	gehe zu: 1
Int16	gehe zu: 1
UInt16	gehe zu: 1
Int24	gehe zu: 1
UInt24	gehe zu: 1
Int32	gehe zu: 1
UInt32	gehe zu: 1
Int64	gehe zu: Ungültig
UInt64	gehe zu: Ungültig
LEB128	gehe zu: 1
ULEB128	gehe zu: 1
AnsiChar / char8_t	□
WideChar / char16_t	□
UTF-8 Codepoint	□ (U+0001)

Bytereihenfolge

☒ Little Endian ☐ Big Endian

„little Endian“ Kodierung
<https://www.computerweekly.com/de/definition/Big-Endian-und-Little-Endian>



Binary File I/O



- Der binäre Datentransfer erfolgt unformatiert, d.h. die Daten werden in der *internen* Darstellung direkt geschrieben bzw. gelesen.

Beispiel: eine float-Zahl (==?) wird nicht als ASCII-Ziffernzeichen formatiert abgespeichert sondern die Bitrepräsentation:

```
02.02.2024 08:56      8 test_double.dat
```

FD test_double.dat																	
Offset (h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Dekodierter Text
00000000	00	00	00	00	00	00	F0	3F									8?

IEEE-754 Floating Point mit „little Endian“ Kodierung -> siehe Übungen

<https://www.h-schmidt.net/FloatConverter/IEEE754.html>

<https://www.computerweekly.com/de/definition/Big-Endian-und-Little-Endian>



Binary File: Schreiben



- Der binäre Datentransfer erfolgt unformatiert, d.h. die Daten werden in der *internen* Darstellung direkt geschrieben bzw. gelesen.
- In C++ wird zum Schreiben
 1. Ein ofstream Objekt mit dem Parameterwert `ios::binary` erzeugt

```
double Zahl = -9.123;  
ofstream ziel("test_double.dat", ios::binary);
```

1

Binary File: Schreiben



- Der binäre Datentransfer erfolgt unformatiert, d.h. die Daten werden in der *internen* Darstellung direkt geschrieben bzw. gelesen.
- In C++ wird zum Schreiben
 1. Ein ofstream Objekt mit dem Parameterwert `ios::binary` erzeugt
 2. Ein Zeiger auf den Beginn des Datenbereiches angegeben, d.h. die Adresse des Bereiches

```
double Zahl = -9.123;  
ofstream ziel("test_double.dat", ios::binary);  
    {&Zahl
```

1

2

Binary File: Schreiben



- Der binäre Datentransfer erfolgt unformatiert, d.h. die Daten werden in der *internen* Darstellung direkt geschrieben bzw. gelesen.
- In C++ wird zum Schreiben
 1. Ein ofstream Objekt mit dem Parameterwert `ios::binary` erzeugt
 2. Ein Zeiger auf den Beginn des Datenbereiches angegeben, d.h. die Adresse des Bereiches
 3. Der Zeiger in den Datentyp `char*` umgewandelt, (das entspricht einem Zeiger auf Bytes)

```
double Zahl = -9.123;  
ofstream ziel("test_double.dat", ios::binary);  
    reinterpret_cast<const char*>(&Zahl)
```

1

2

3

Binary File: Schreiben



- Der binäre Datentransfer erfolgt unformatiert, d.h. die Daten werden in der *internen* Darstellung direkt geschrieben bzw. gelesen.
- In C++ wird zum Schreiben
 1. Ein ofstream Objekt mit dem Parameterwert `ios::binary` erzeugt
 2. Ein Zeiger auf den Beginn des Datenbereiches angegeben, d.h. die Adresse des Bereiches
 3. Der Zeiger in den Datentyp `char*` umgewandelt, (das entspricht einem Zeiger auf Bytes)
 4. Mit `write()` in einen ofstream geschrieben

```
double Zahl = -9.123;  
ofstream ziel("test_double.dat", ios::binary);  
ziel.write( reinterpret_cast<const char*>(&Zahl), sizeof(Zahl));
```

4

1

2

3

Binary File: Schreiben



- Der binäre Datentransfer erfolgt unformatiert, d.h. die Daten werden in der *internen* Darstellung direkt geschrieben bzw. gelesen.
- In C++ wird zum Schreiben
 1. Ein ofstream Objekt mit dem Parameterwert `ios::binary` erzeugt
 2. Ein Zeiger auf den Beginn des Datenbereiches angegeben, d.h. die Adresse des Bereiches
 3. Der Zeiger in den Datentyp `char*` umgewandelt, (das entspricht einem Zeiger auf Bytes)
 4. Mit `write()` in einen ofstream geschrieben

Anzahl der zu schreibenden Bytes

```
double Zahl = -9.123;  
ofstream ziel("test_double.dat", ios::binary);  
ziel.write( reinterpret_cast<const char*>(&Zahl), sizeof(Zahl));
```

Binary File: Schreiben



- Der binäre Datentransfer erfolgt unformatiert, d.h. die Daten werden in der *internen* Darstellung direkt geschrieben bzw. gelesen.
- In C++ wird zum Schreiben
 1. Ein ofstream Objekt mit dem Parameterwert `ios::binary` erzeugt
 2. Ein Zeiger auf den Beginn des Datenbereiches angegeben, d.h. die Adresse des Bereiches
 3. Der Zeiger in den Datentyp `char*` umgewandelt, (das entspricht einem Zeiger auf Bytes)
 4. Mit `write()` in einen ofstream geschrieben
 5. Stream mit `close()` geschlossen.

```
double Zahl = -9.123;  
ofstream ziel("test_double.dat", ios::binary);  
ziel.write( reinterpret_cast<const char*>(&Zahl), sizeof(Zahl));  
ziel.close();
```

Binary File: Lesen



- Der binäre Datentransfer erfolgt unformatiert, d.h. die Daten werden in der *internen* Darstellung direkt geschrieben bzw. gelesen.
- In C++ wird zum Lesen
 1. Ein ifstream Objekt mit dem Parameterwert `ios::binary` erzeugt
 2. Ein Zeiger auf den zu lesendem Datentyp angegeben, d.h. die Adresse des Bereiches
 3. Der Zeiger in den Datentyp `char*` umgewandelt, (das entspricht einem Zeiger auf Bytes)
 4. Mit `read()` vom ifstream in den Speicher geschrieben
 5. Stream mit `close()` geschlossen.

```
double Zahl;  
ifstream quelle("test_double.dat", ios::binary),  
quelle.read( reinterpret_cast<char*>(&Zahl), sizeof(Zahl));  
quelle.close();
```

4

1

3

5

2

Binary File: Lesen



- Der binäre Datentransfer erfolgt unformatiert, d.h. die Daten werden in der *internen* Darstellung direkt geschrieben bzw. gelesen.
- In C++ wird zum Lesen
 1. Ein ifstream Objekt mit dem Parameterwert `ios::binary` erzeugt
 2. Ein Zeiger auf den zu lesendem Datentyp angegeben, d.h. die Adresse des Bereiches
 3. Der Zeiger in den Datentyp `char*` umgewandelt, (das entspricht einem Zeiger auf Bytes)
 4. Mit `read()` vom ifstream in den Speicher geschrieben
 5. Stream mit `close()` geschlossen.

```
double Zahl;  
ifstream quelle("test_double.dat", ios::binary);  
4  quelle.read( reinterpret_cast<char*>(&Zahl), sizeof(Zahl));  
5  quelle.close();
```

Beim Schreiben nach `&Zahl` wird die Zahl geändert, daher kein `const char*`

Binary File I/O



- Der binäre Datentransfer erfolgt unformatiert, d.h. die Daten werden in der *internen* Darstellung direkt geschrieben bzw. gelesen.

Benutzen des Rückgabewertes bei Erstellen des Streams, um Fehler abzufangen:

```
double Zahl = -9.123;
ofstream ziel("test_double.dat", ios::binary);
if (!ziel){
    cout << "could not open file for writing \n";
    return -1;
}
ziel.write( reinterpret_cast<const char*>(&Zahl), sizeof(Zahl));
ziel.close();
```





Code aus der Vorlesung



// cast

```
float b = 0.999999f;  
int a = b;  
a = int(b);  
a = (int)b;  
a = static_cast<int>(b);  
cout << a << endl;
```

// reinterpret_cast

```
unsigned int a = pow(2, 31);  
int b = 21;  
float c = 1.4;  
int* ptr = &b;  
*ptr = 10 + *ptr;  
ptr = &b;  
b = *ptr + a;  
float* p = reinterpret_cast<float*>(&a);  
c = *p;  
cout << "1:" << &c << endl;  
cout << "2:" << &ptr << endl;  
cout << *p << endl;
```



Code aus der Vorlesung

```
// reinterpret_cast
```

```
unsigned int x = pow(2, 31);  
float* z;  
z = reinterpret_cast<float*>(&x);  
cout << *z << endl;
```



Code aus der Vorlesung



```
#include <fstream>
#include <iostream>
#include <string>

using namespace std;

int main() {
    // Text Dateien I/O
    ofstream MeineDatei;
    MeineDatei.open("test.txt");
    MeineDatei << "das ist ein Test" << endl;
    MeineDatei.close();

    ifstream MeineInputDatei("test.txt");
    string zeile;
    while (MeineInputDatei.good()) {
        getline(MeineInputDatei, zeile);
        cout << zeile;
    }
}
```



Code aus der Vorlesung



```
#include <fstream>
#include <iostream>
#include <string>
using namespace std;

int main() {
    unsigned int x = pow(2, 31);

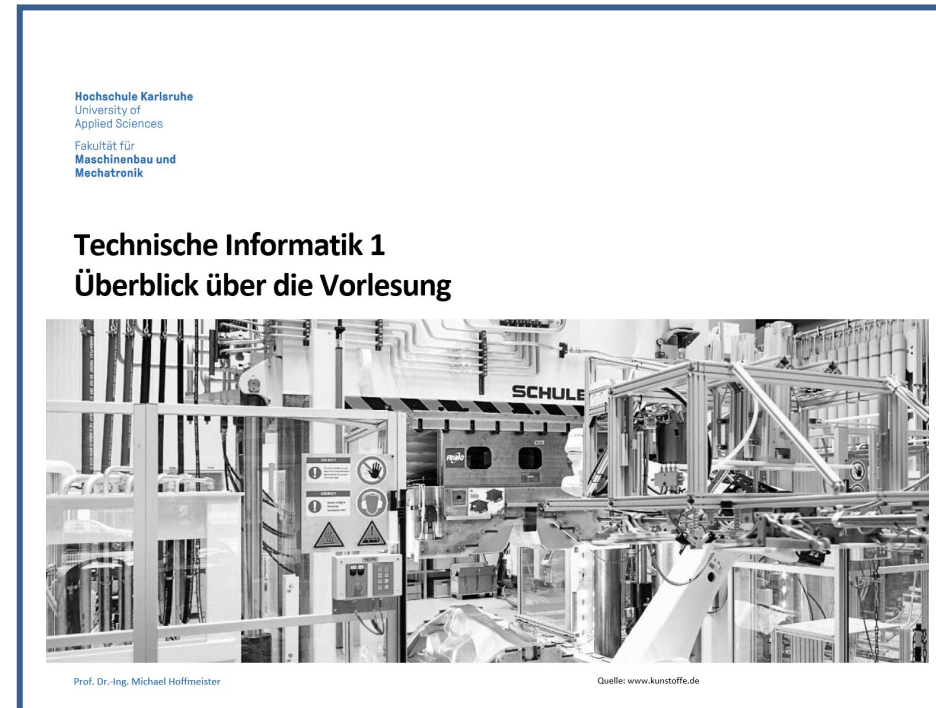
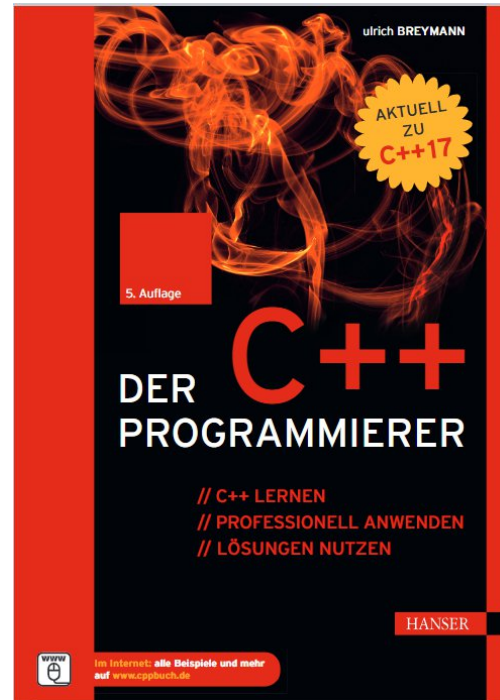
    // Binaer Dateien I/O (Schreiben)
    ofstream MeineBinaerDatei;
    MeineBinaerDatei.open("test.bin" , ios::binary);
    MeineBinaerDatei.write(reinterpret_cast<char*>(&x), 4);
    MeineBinaerDatei.write(reinterpret_cast<char*>(&x), 4);
    MeineBinaerDatei.close();
```

```
// Binaer Dateien I/O (Lesen)
ifstream MeineBinaerDateiInput;
MeineBinaerDateiInput.open("test.bin", ios::binary);
unsigned char Mem;
float Memf;

while (MeineBinaerDateiInput.good()) {
    MeineBinaerDateiInput.read(reinterpret_cast<char*>(&Mem), 1);
    cout << int(Mem) << endl;
    MeineBinaerDateiInput.read(reinterpret_cast<char*>(&Memf), 4);

    cout << Memf << endl;
    return 0;
}
```





Prof. Dr.-Ing. Michael Hoffmeister

Aufzählungs- oder Enumerationstypen



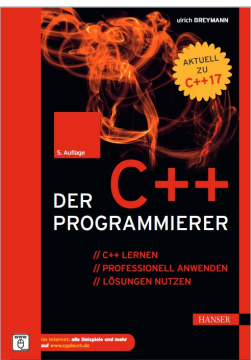
Häufig gibt es nicht-numerische Wertebereiche. So kann zum Beispiel ein Wochentag nur die Werte Sonntag, Montag, Dienstag, Mittwoch, Donnerstag, Freitag und Samstag annehmen. Oder ein Farbwert in einem C++-Programm soll nur den vier Farben Rot, Grün, Blau, und Gelb entsprechen. Eine mögliche Hilfskonstruktion wäre die Abbildung auf den Datentyp `int`:

```
int eineFarbe;           // rot = 0
                        // grün = 1
                        // blau = 2
                        // gelb = 3

int einWochentag;        // Sonntag = 0
                        // Montag = 1 usw.
```

Dieses Verfahren hätte einige Nachteile:

- Die Bedeutung muss im Programm als Kommentar festgehalten werden.
- Zugeordnete Zahlen sind nicht eindeutig: 0 kann rot, aber auch Sonntag bedeuten, und 1 kann für grün oder auch Montag stehen.
- Schlechter Dokumentationswert, zum Beispiel `if(eineFarbe == 2) ...`. Hieraus ist nicht zu ersehen, welche Farbe gemeint ist. Oder: `if(eineFarbe == 5) ...`. Der Wert 5 ist undefiniert!



Aufzählungs- oder Enumerationstypen



Der neue Datentyp Farbtyp wird deklariert:

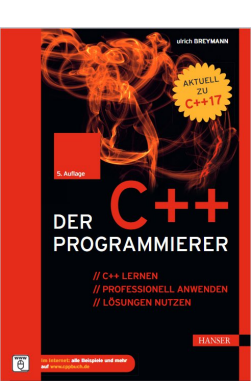
```
enum class Farbtyp {rot, gruen, blau, gelb};
```

Der neue Datentyp Wochentag wird deklariert:

```
enum class Wochentag {sonntag, montag, dienstag, mittwoch,  
                     donnerstag, freitag, samstag};
```

Wenn der Datentyp erst einmal bekannt ist, können weitere Variablen definiert werden:

```
Farbtyp gewaehlteFarbe;           // Definition  
Wochentag derFeiertag, einWerktag; // Definitionen  
Wochentag heute = Wochentag::dienstag; // Definition + Initialisierung
```



Aufzählungs- oder Enumerationstypen



Der neue Datentyp Farbtyp wird deklariert:

```
enum class Farbtyp {rot, gruen, blau, gelb};
```

Der neue Datentyp Wochentag wird deklariert:

```
enum class Wochentag {sonntag, montag, dienstag, mittwoch,  
                     donnerstag, freitag, samstag};
```

Wenn der Datentyp erst einmal bekannt ist, können weitere Variablen definiert werden:

```
Farbtyp gewaehlteFarbe;           // Definition  
Wochentag derFeiertag, einWerktag; // Definitionen  
Wochentag heute = Wochentag::dienstag; // Definition + Initialisierung
```

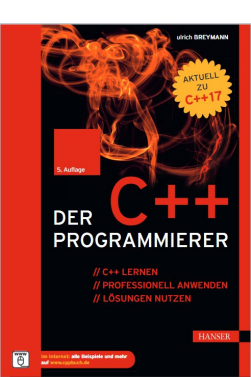
// Abweichung von der Standardeinstellung 0, 1, 2, 3 ...:

// Deklaration:

```
enum class Farbtyp {rot = 0, gruen = 1, blau = 2, gelb = 4};
```

// Deklaration mit Variablendefinition:

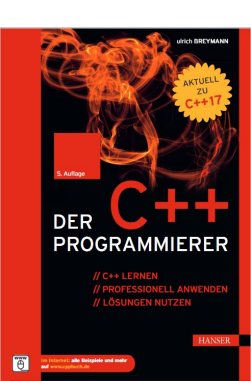
```
enum class Palette { weiss = 0, grau = 1, braun = 2, amber = 4, lila = 8  
                   } mischung;
```



Aufzählungs- oder Enumerationstypen



```
Farbtyp gewaehlteFarbe;           // Definition
Wochentag derFeiertag, einWerktag, // Definitionen
heute = Wochentag::dienstag;      // Definition + Initialisierung
int j = Wochentag::dienstag;      // Fehler! Typ nicht kompatibel
int i = static_cast<int>(Wochentag::dienstag); // jetzt ok
heute = Wochentag::montag;        // richtig
heute = i;                       // Fehler, Datentyp inkompatibel
heute = static_cast<Wochentag>(i); // erlaubt, aber...
// Falls die Variable i einen Wert hat, der nicht einem der Werte des
// Aufzählungstyps entspricht, ist der Wert der Variablen heute undefiniert!
```



Aufzählungs- oder Enumerationstypen



class weglassen?

Das Schlüsselwort `class` wegzulassen ist weiterhin gültiges C++, nur ist die Typsicherheit eingeschränkt, weil implizite Umwandlungen nach `int` möglich sind. Ein Beispiel:

```
enum Farbtyp { rot, gruen, blau, gelb };           // ohne class  
int i = rot + blau;                               // nun möglich
```

Außer der impliziten Umwandlung von `rot` und `blau` nach `int` fällt auf, dass die Typqualifizierung `Farbtyp::` fehlen kann.



Tipp

Wegen der erhöhten Typsicherheit wird empfohlen, `enum class` zu verwenden und nur in begründeten Ausnahmefällen darauf zu verzichten.

